2. Python SQLite Game Shop POS

Contents

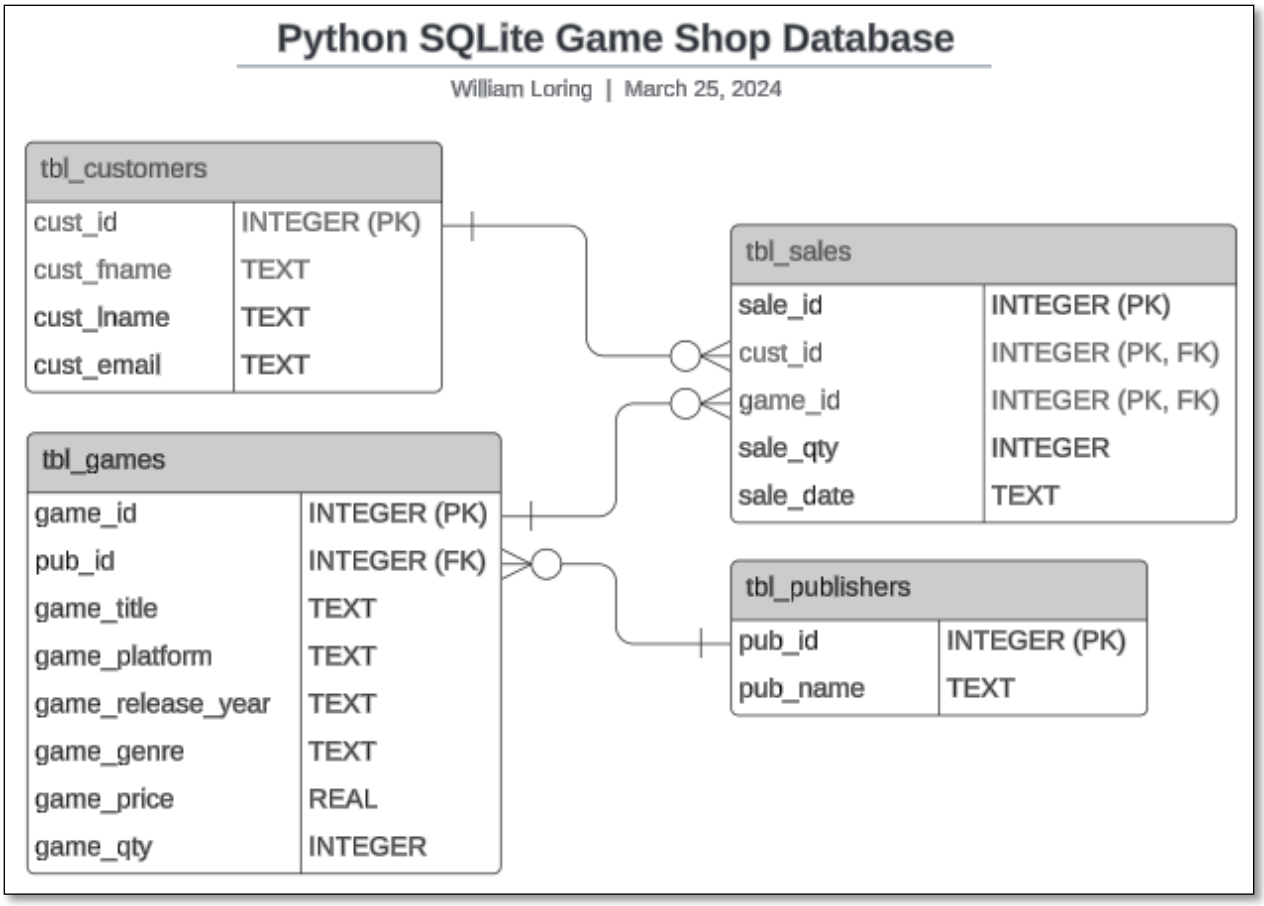
2. Python SQLite Game Shop POS 1
Main App Modification 5
Menu Re Work 6
Assignment Submission.....8

Time required: 90 minutes

A humorous and educational video on Python SQLite: SQLite in Python

- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.
- Remember to modify to create your own database

We redesigned our database in Part 1. Let's take a look at how to convert the ERD to a table.



Based on our new ERD, we need to add a sales table as a bridge table.

Create a new Python file named: **db_sales.py**

```python
"""
    Name: db_sales.py
    Author: William Loring
    Created: 10/07/24
    CRUD module for game sales transactions
"""
# Import sqlite3 database library
import sqlite3

DATABASE = "game_shop.db"

# --------------------- SQL STATEMENTS ------------------------------------- #
# SQL statements are text. SQL queries can be very long.
# You can create a SQL statement and assign it to a string variable.

# ------------------------- CREATE TABLE ---------------------------------- #
# Create a table to store sales transactions
# Each sale record has a unique identifier, customer ID, game ID,
# quantity sold, and sale date
CREATE_TABLE = """
    CREATE TABLE IF NOT EXISTS tbl_sales (
    sale_id INTEGER PRIMARY KEY,  -- Unique identifier for each game
    cust_id INTEGER,              -- Foreign key referencing customers table
    game_id INTEGER,              -- Foreign key referencing games table
    sale_qty INTEGER              -- Number of copies sold
    sale_date INTEGER,            -- Date of sale
    FOREIGN KEY (cust_id) REFERENCES tbl_customers(cust_id),
    FOREIGN KEY (game_id) REFERENCES tbl_games(game_id)
    CONSTRAINT fk_customer
        FOREIGN KEY (cust_id)
        REFERENCES tbl_customers(cust_id)
        ON DELETE CASCADE
    CONSTRAINT fk_game
        FOREIGN KEY (game_id)
        REFERENCES tbl_games(game_id)
        ON DELETE CASCADE
);
"""
```

Here is a breakdown of the tbl_sales CREATE_TABLE SQL.

1. **Table Creation Statement**:

```
CREATE TABLE IF NOT EXISTS tbl_sales (
```

- This line starts the creation of a new table named **tbl_sales** if it does not already exist.

2. **Column Definitions**:
   - **sale_id INTEGER PRIMARY KEY**: Defines **sale_id** as an integer and the primary key, ensuring each sale has a unique identifier.
   - **cust_id INTEGER**: Defines **cust_id** as an integer, which will act as a foreign key referencing the **tbl_customers** table.
   - **game_id INTEGER**: Defines **game_id** as an integer, which will act as a foreign key referencing the **tbl_games** table.
   - **sale_qty INTEGER**: Defines **sale_qty** as an integer, representing the number of copies sold.
   - **sale_date INTEGER**: Defines **sale_date** as an integer, representing the date of the sale.
3. **Foreign Key Constraints**:
   - **FOREIGN KEY (cust_id) REFERENCES tbl_customers(cust_id)**: Establishes a foreign key constraint on cust_id, linking it to the cust_id column in the tbl_customers table.
   - **FOREIGN KEY (game_id) REFERENCES tbl_games(game_id)**: Establishes a foreign key constraint on game_id, linking it to the game_id column in the tbl_games table.
4. **Additional Constraints**:

```
CONSTRAINT fk_customer
    FOREIGN KEY (cust_id)
    REFERENCES tbl_customers(cust_id)
    ON DELETE CASCADE:
```

   - Adds a named constraint **fk_customer** that enforces the foreign key relationship with **tbl_customers** and specifies that if a referenced customer is deleted, the corresponding sales records will also be deleted (ON DELETE CASCADE).

```
CONSTRAINT fk_game
    FOREIGN KEY (game_id)
    REFERENCES tbl_games(game_id)
    ON DELETE CASCADE:
```

- Adds a named constraint **fk_game** that enforces the foreign key relationship with **tbl_games** and specifies that if a referenced game is deleted, the corresponding sales records will also be deleted (ON DELETE CASCADE).

This SQL code ensures that the tbl_sales bridge table is properly linked to the tbl_customers and tbl_games tables, maintaining referential integrity and cascading deletions when necessary.

Main App Modification

Make the following changes to your main application by adding the reference to the new **db_sales** module and **db_sales.create_table()** method.

```python
"""
    Name: game_shop_pos_1.py
    Author: William Loring
    Created: 02/16/25
    Use SQLite with Python
"""

import tabulate
import db_app
import db_games
import db_publishers
import db_sales

MENU_PROMPT = """------  Game Shop Point of Sale  ------
(1) Add publisher
(2) Display all publishers
(3) Add new game
(4) Display all games
(5) Display all games and publishers
(6) Delete publisher
(7) Delete game
(9) Exit
Your selection: """


def main():
    db_publishers.create_table()
    db_games.create_table()
    db_sales.create_table()
    menu()
```

Menu Re Work

We are going to have a lot more choices in this new database. Time to redesign our menu system.

```python
MENU_PROMPT = """------  Game Shop Point of Sale  ------
(1) Table Maintenance
(2) Sales Transactions
(3) Reports
(9) Exit
Your selection: """

TABLE_MAINTENANCE_PROMPT = """------  Table Maintenance  ------
(1) Insert Publisher
(2) Insert Game
(3) Delete Publisher
(4) Delete Game
(9) Exit
"""

SALES_TRANSACTIONS_PROMPT = """------  Sales Transactions  ------
(1) Add Sale
(2) Display Sales Transactions
(3) Delete Sales Transaction
(9) Exit
"""

REPORTS_PROMPT = """------  Reports  -------
(1) Sales by Publisher
(2) Sales by Game
(3) Sales by Genre
(4) Sales by Platform
(5) Display Publishers
(6) Display Games
(7) Display Publishers & Games
(8) Display Sales Transactions
(9) Exit
"""
```

Move your existing menu items into the new menu structure using the nested pattern shown.

```python
def menu():
    while True:
        user_input = input(MENU_PROMPT)

    # ---------------------- TABLE MAINTENANCE ---------------------------- #
        if user_input == "1":

            # ---------------------- INSERT PUBLISHER ---------------------- #
            user_input = input(TABLE_MAINTENANCE_PROMPT)
            if user_input == "1":
                display_all_publishers()
                # Get publisher data from user
                pub_name = input("Enter publisher name: ")
                pub_country = input("Enter publisher country: ")

                # Add publisher to database
                db_publishers.add_record(
                    pub_name,
                    pub_country
                )

                display_all_publishers()
```

As this project continues, we will add the SQL and code needed to finish out the menu items.

Stay tuned for more Fun with SQLite!

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.